Games@Cloud

Tiago Deane (ist1103811), Tomás Silva (ist195684) and Valdemar Armindo (ist1105668)

Instituto Superior Técnico – Universidade de Lisboa
Master in Computer Science and Engineering (MEIC)
Cloud Computing and Virtualization

***Abstract*—This project presents the development of Games@Cloud, a cloud-based system created to run demanding tasks based on classic games like Capture the Flag, Fifteen Puzzle, and Game of Life. The system runs on Amazon Web Services (AWS) and uses two types of workers: EC2 instances, which are slower to start but cheaper per task, and Lambda functions, which start quickly but cost more. Each user request is handled by a central Load Balancer, which decides whether to use an EC2 or a Lambda based on how complex the task is. This complexity is estimated using data collected by a Javassist tool that analyzes how much work the code performs. These metrics are saved in DynamoDB and used to improve future decisions. To manage resources, an Auto-Scaler monitors the system and increases or decreases the number of EC2 instances depending on how busy the system is. All components are automated to make deployment and resource management easier. In the this final version, the system is able to handle different types of requests efficiently, keeping a good balance between performance and cost.**

***Index Terms*—cloud computing, auto-scaling, load balancing, AWS (Amazon Web Services), instrumentation.**

I. Introduction

THE growing demand for scalable and adaptive computing has made cloud-based architectures a natural fit for applications with variable workloads according to [1]. These systems benefit from being able to dynamically adjust resources to meet changing levels of demand, ensuring performance while minimizing cost. The Games@Cloud project explores this paradigm through the implementation of a cloud native system capable of executing a set of CPU-intensive tasks inspired by simple but computationally demanding games. The system supports three distinct types of workloads: Capture the Flag, where agents navigate a board to capture the opponent's flag; Fifteen Puzzle, which involves solving a shuffled sliding-tile puzzle; and Game of Life, which simulates cellular evolution based on Conway's rules. Each game introduces a different level of complexity depending on its input parameters (e.g., board size, iteration count, number of agents), making the computational cost of requests highly variable. As such, the system must not only be able to execute these tasks correctly, but also adapt its resource usage dynamically to ensure efficiency and responsiveness. The objective of the project is to design and implement an elastic cloud service that can receive requests to run these games and decide based on task complexity and system state how to execute them using a combination of virtual machines (EC2) and serverless functions (AWS Lambda).

This final report presents the complete design and implementation of the system. It begins with an overview of the architecture and its main components. Section 3 discusses the instrumentation process and the metrics collected. Section 4 describes the data structures used across the system. Section 5 details how request cost estimation is performed based on historical and real-time data. Section 6 presents the auto-scaling algorithm and its integration with the Load Balancer. Section 7 addresses basic aspects of fault-tolerance, and Section 8 concludes with final remarks and reflections on the system's performance and scalability.

II. Architecture

Figure 1 illustrates the high-level architecture of the *Games@Cloud* system, reflecting its **modular** and **distributed** design. The diagram emphasizes the interaction between the system's **core components** and how they cooperate to process user requests efficiently in an **elastic cloud environment**.

At the top of the architecture, **clients** issue game-related HTTP requests. These requests are received by the **Load Balancer (LB)**, which is deployed on an EC2 instance. The LB is the system's **single entry point** and is responsible for **estimating the complexity** of each request. This estimation is based on **instrumentation metrics** retrieved from the **DynamoDB tables**, which contain both **historical** and **current execution data**.

After estimating complexity, the LB dispatches the request to the appropriate compute backend:

- **Low-complexity requests** are routed to **AWS Lambda functions**, which provide **low-latency responses** and avoid unnecessary EC2 boot-up time.
- **High-complexity requests** are routed to the **Worker Pool**, a dynamic set of EC2 instances running a **multi-threaded Java webserver** instrumented with **Javassist**.

To manage elasticity, the LB runs a background **Auto-Scaler thread** that periodically checks system load and adjusts the number of active EC2 workers. **Scaling decisions** are based on the **total estimated load** of running requests, ensuring that resource usage remains proportional to demand. At the data level, **DynamoDB** plays a central role. One table persistently stores **extracted metrics** from completed requests, while the other keeps track of **ongoing requests**, supporting **real-time load estimation** and **request distribution**.

This architecture supports **parallel execution** through **multithreading** at the worker level and allows **independent horizontal scaling** of the worker pool, making the system both **efficient** and **cost-aware**. The folder "/scripts" contains scripts to create the webserver AMI, launch the load balancer and register the necessary lambda functions.

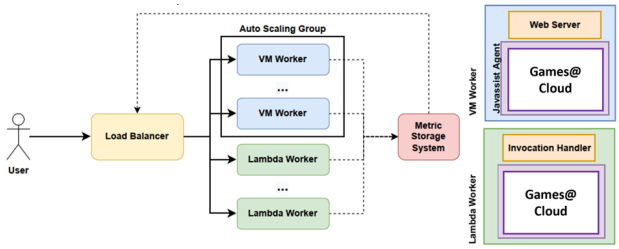


Fig. 1. Architecture of Games@Cloud.

III. Instrumentation Metrics

The webserver is instrumented with a Javassist Agent, MetricsExtractor, to extract the number of instructions and the number of memory accesses of each executed request. Using execution time as a metric isn't an option, as request metrics need to be applicable regardless of what type of instance is executing the request. To properly estimate each request's complexity, we store their number of instructions and number of memory accesses. This is because instructions with memory accesses take much longer than ones without, and there may be workloads with similar instruction counts but vastly different amounts of memory accesses.

On compile-time, the agent stores on each basic block the number of instructions with memory accesses that it contains. At runtime, before each request, the tool resets the metrics extracted from previous requests. Then, at the start of each basic block, it adds the number of instructions and the number of memory accesses contained by said block, thus keeping track of the metrics for that specific request. The tool uses threads, so multiple requests can be executed simultaneously and metrics are extracted separately for each one. At the end of each request's execution, the webserver extracts the metrics stored by the tool and writes them to DynamoDB, along with the request's arguments and the handler's name.

IV. Data Structures

The request information is written to DynamoDB, enabling all running instances to access the data regardless of which machine originally processed the request. We keep two DynamoDB tables:

1) CNVRequestMetrics:

```
{
  "args": "[arg1,...]", // primary key
  "handler": "HandlerName",
  "nInstructions": "1500",
  "nMemAccesses": "500"
}
```

2) CNVOngoingRequests:

```
{
  "instance_id": "vm-123", // primary key
  "request_id": "abc123", // sort key
  "handlerName": "HandlerName",
  "nInstructions": "1500",
  "nMemAccesses": "500"
}
```

The first table, CNVRequestMetrics, stores metrics from past requests using a list of arguments as the primary key. Since requests with the same arguments always have the same metrics, we can simply look for that set of arguments on the CNVRequestMetrics table and then return the request's number of instructions and number of memory accesses. The name of the handler who executed the request is also stored, in order to help parse the arguments and give more information about the request. The metrics (nInstructions and nMemAccesses) are stored separately, so that models can be trained on them for better complexity estimation.

It is worth mentioning that using the arguments by themselves as the primary key only works because each handler receives a different number of arguments, so there's no conflict regarding primary keys. However, if a new handler were to be added with the same number and type of arguments as another handler, we would have to make the handler's name a sort key in order to avoid conflicts. With the three handlers that we were given, however, this is not necessary.

The second table, CNVOngoingRequests, stores the ongoing requests being executed by running instances, using the corresponding EC2 instance's ID as the primary key and a unique request ID as sort key. The request's metrics are stored to allow for load balancing: the stored metrics for each request (and the instance executing it) are used as a measure of the current load on that specific instance. For cases where the request being executed has never been seen before (i.e. there was no entry on the CNVRequestMetrics table with that set of arguments), the metrics stored on this table are an estimation. The details of the cost estimation process are explained in the next section. Upon request completion (or failure), the corresponding entry is removed from the table. The request failure scenario is explained in more detail in Section VIII.

V. Request Cost Estimation

Our cost estimation grows more precise as our system receives and stores the metrics from a wider variety of requests. Upon receiving a new request, we build a searching key from the input values. We use this key to confirm if the respective input was already received and stored. If this is the case, we use the stored number of instructions and memory accesses to compute the cost for the request. In case those input values were never received, we estimate them.

Primarily, this estimation is done per handler. This means that we do not use data from the CaptureTheFlagHandler to perform estimations on a request to the GameOfLifeHandler. The estimation is performed using KNN (with k = 3). Detailed discussions of this method can be found in the literature [2]. As our system receives and stores a larger variety of requests, the KNN estimation becomes increasingly more precise. Finally, we apply a multiple linear regression to the values provided by the KNN model to estimate the execution time.

We decided to separate the estimates based on the handler because they have differences, specifically in the ratio of memory accesses per total executed instructions. As we show in Figure 2, this relation is linear and differs between different handlers. Although this ratio is quite similar in CaptureTheFlag and FifteenPuzzle, we decided to separate them to keep our logic consistent and account for any deviations specific to each case.

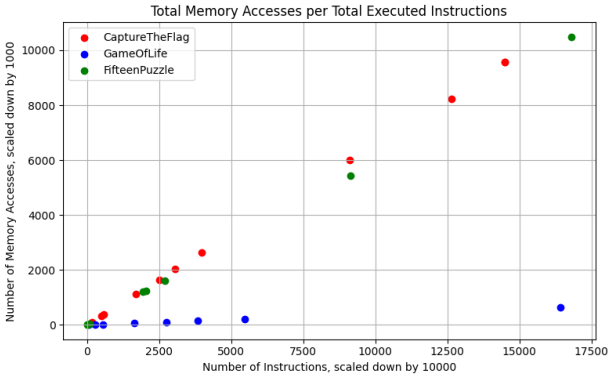


Fig. 2. Total Memory Accesses per Total Executed Instructions for all request types

We used these benchmarked values to train a multiple linear regression model for each request type. Since this estimation is going to be used to decide whether to send requests to an EC2 instance or a Lambda function, the target execution times were based on the ”billed duration” of Lambda function executions: we executed each request five times, and then used an average of those values. From Figures 3 to 5, it is clear that execution time grows linearly with the number of memory accesses for each request type.

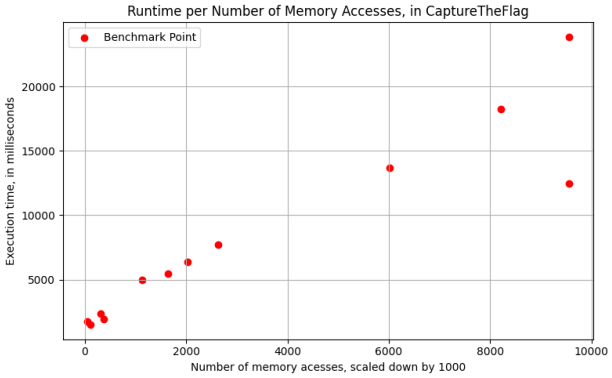


Fig. 3. Runtime per Number of Memory Accesses in CaptureTheFlag requests

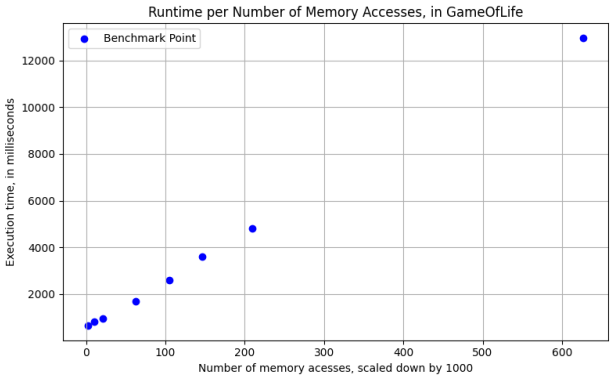


Fig. 4. Runtime per Number of Memory Accesses in GameOfLife requests

So, for each request type, we predict the execution time based on the estimated (or fetched) number of instructions and memory accesses applied to the corresponding regression. We compare it with a defined threshold and send the request to an EC2 instance if the predicted execution time is above it, or to a Lambda instance if it is below.

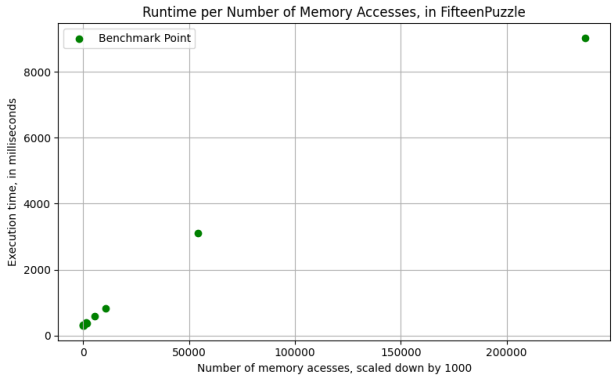


Fig. 5. Runtime per Number of Memory Accesses in FifteenPuzzle requests

We take into consideration an error margin on our time estimate, specifically the Root Mean Squared Error, as it is more adequate to deal with large errors. These are our main concerns, as we wish to avoid sending large requests that we wrongly deemed low to Lambda instances. As part of this effort to avoid Lambda function timeouts, we also included outlier values caused by cold starts in Lambda function executions when training the models, reducing the risk of timeouts being caused by cold starts.

It is also important to note that for the obtained regression for CaptureTheFlag, our computed error was larger compared to those of the other handler types. While testing, this led to most CaptureTheFlag requests to be executed in EC2 instances, and not in Lambdas. To accomodate this, we decided to make a small increase to the threshold value for these requests, and not consider the error margin. We experimentally confirmed that this change allows smaller CaptureTheFlag requests to be executed in Lambdas, while larger requests are still routed to EC2 instances.

VI. Task Scheduling Algorithm (Load Balancing)

As previously mentioned, we use the CNVOngoingRequests table to store information on the requests that are being executed. The load balancer is responsible for registering and deregistering requests from the table. The stored information allows us to compute the time complexity of the requests that are being executed. The handler identification allows us to apply the respective linear regression to the instructions and memory accesses. This allows us to keep a precise estimation across the whole process.

We considered the total workload weight of an instance as the sum of the time complexity of all the requests that the instance is executing. So, every time a new request is forwarded to an EC2 instance, we simply verify which of the running instances has the lowest total load. From the decided instance ID (to which we have access by the CNVOngoingRequests table), we convert it into the IP of the respective instance, and forward the request.

If a request is received when multiple instances do not have any work, the request can be forwarded to any of the instances. If, on the contrary, a request is received when no instance

is active, the load balancer keeps it on hold, waiting for the AutoScaler to launch a new EC2 instance.

We decided to always send lighter requests to Lambda instances. Since Lambdas are billed per request, we considered that it was not worth it to fill them with high workloads. Also, since Lambdas can suffer from cold starts, we can compensate for any latency with quick executable requests. This means that, if our system receives a large batch of light requests, these will all be routed to Lambda instances (assuming no failures occur). On the other hand, we mostly enforce that new EC2 instances may only be launched to deal with large requests. The only exception to this is if Lambda requests fail. We defined a maximum number of three retries for Lambdas. This means that small requests may end up being routed to an EC2 instance, but only if the respective Lambda fails three consecutive times. This routing follows the same principle mentioned above of using the instance with the lowest total workload.

The following is a simplified pseudocode that illustrates how the load balancing algorithm operates:

```
handle(request):
  metrics = estimateCost(request.args)
  useLambda = redirectToLambda(metrics)
  handled = false
  if (useLambda):
    handled = sendToLambdaWithRetry(request)
  while not handled:
    destinationId = getInstanceWithLowestLoad()
    registerRequest(destinationId, request, metrics)
    response = sendToEC2Instance(request,
                                 destination)
    deregisterRequest(destinationId, request)
    if response != null: handled = true
```

VII. Auto-Scaling Algorithm

The AutoScaler is a thread on the Load Balancer instance that runs every 30 seconds. During its activity, the AutoScaler keeps track of three instance states:

- *A set of instances that were "signaled" as starting*: every time a new instance is launched (during a Scale-Up event), it is added to this set. The next cycle is responsible for verifying that this launch was successfully completed (by checking if the respective IP address is reachable and receives requests on port 8000, as expected). If this is the case, it also adds the instance ID to a set of active instances and removes it from the original. If this is not the case, the instance is left to be reevaluated on the next cycle.
- *A set of instances that were "signaled" for termination*: every time a new instance is marked for deletion (during a Scale-Down where the selected instance to terminate is not idle), it is added to this set. The AutoScaler verifies if the instances in this set are idle. An instance is considered idle if, according to the CNVOngoingRequests table, it is not executing any requests at that moment. If this is the case, the instance is terminated and removed from this set.
- *A set of instances that are "signaled" as active*: on each cycle, the AutoScaler performs an activity check on each instance it considers active. It performs the same type of activity check as in the instances marked as starting. In this case, it tries to connect with the instances. If this process fails along the various checks, the AutoScaler removes the critical instance from the list of active instances and decreases the active instance counter.

The AutoScaler leverages CloudWatch to obtain the average CPU utilization of all active instances, and is able to trigger **Scale-Up** and **Scale-Down** events to deal with the current load on the system, with a five-minute cooldown period between any two scaling events.

During **Scale-Ups**, the AutoScaler starts launching a new instance, stores its attributed ID in the turningOnInstanceIds set, and increments its count of active instances. Scale-ups are performed when the average overall CPU usage is over 70%, or when the number of active instances is below the minimum threshold (of at least one active instance).

During **Scale-Downs**, the AutoScaler decides which active instance is executing the lowest workload (using the same method as previously explained in Section VI). It performs all the necessary local changes, such as decreasing the local active instance counter, and removing the selected instance from the active instances' list. If the selected instance is idle, it is immediately terminated. If it is not idle (still executing pending requests), it is added to the set of instances "signaled" for termination. Scale-downs are performed when the average overall CPU usage is below 30%.

To deal with failed requests, the load balancer can call the AutoScaler on demand to perform an activity check on the instance who was responsible for the request, who will in turn remove that instance from the set of active instances if it doesn't respond.

The following is a simplified pseudocode illustrating an auto-scaling cycle:

```
evaluateAndScale():
  for (instance waiting for deletion):
    terminateInstanceIfIdle(instance)
  for (instance turning on or already active):
    checkInstanceActivity(instance)

  if (time since last scaling event < 300 seconds):
    return

  averageCPU = getAverageCPUUtilization()
  if (currentCapacity < minimumCapacity):
    lastScalingEvent = now
    scaleUp()
  else if (averageCpu > SCALE_UP_THRESHOLD &&
           currentCapacity < maximumCapacity):
    lastScalingEvent = now
    scaleUp()
  else if (averageCpu < SCALE_DOWN_THRESHOLD &&
           currentCapacity > minimumCapacity):
    lastScalingEvent = now
    scaleDown()
```

VIII. Fault-Tolerance

To ensure high availability and resilience, our system employs multiple techniques to deal with faults.

As mentioned previously, low-complexity requests are first sent to Lambda functions. Lambdas will attempt execution three times. If all three attempts fail, either due to timeouts or other failures, the request is then redirected to an EC2 instance.

If at any moment a request is to be forwarded to an EC2 instance but there are no active instances, the load balancer will hold that request and wait for new instances to be ready before continuing. If a request forwarded to an EC2 instance fails, the load balancer leverages the AutoScaler to check the respective instance's status. The AutoScaler performs the activity check described in Section VII. If it deems the instance unusable, it is removed from the list of active instances. After this check is performed, the load balancer once again fetches the EC2 instance with the lowest total workload and forwards the affected request to it. During this whole process, it is also the load balancer who manages the registering and deregistering of requests in the CNVOngoingRequests table.

The only fault our system is not capable of handling is a failure of the load balancer itself. As a fail-safe, however, the load balancer is configured to clean up all entries of ongoing requests from the CNVOngoingRequests table upon being shut down, as well as call the AutoScaler to terminate all EC2 instances.

Figure 6 shows a sequence diagram representing how the system handles requests and deals with faults.

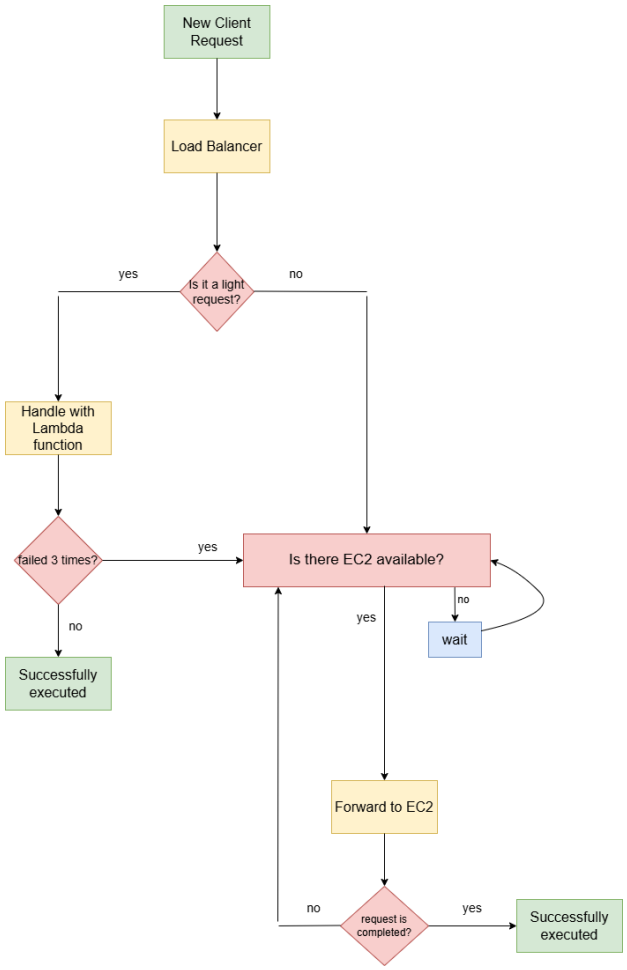


Fig. 6. Execution Sequence Diagram

IX. Conclusion

To conclude, we managed to develop a fully functional cloud system, capable of receiving and handling different requests. The entry component is the LoadBalancer. By using memory accesses and the total number of instructions as complexity metrics, it is capable of estimating the cost of requests and deciding the better course of action for handling them. Practical request handling is done by resorting to both Lambda functions and EC2 instances, both with tradeoffs in costs and latency. We opted for a conservative approach, where a bit of latency was preferred to expensive execution costs.

This system is supported by an AutoScaler, which not only takes action on scaling up and down the overall EC2 number, but also keeps track of any failing instances and ensures that the system is never out of alternatives to deal with requests.

Relevant data for decision making, such as the requests' metrics and ongoing requests, is stored in DynamoDB.

As possible sources for direct improvement, we highlight the capacity to tolerate LoadBalancer failure and a more fine-grained routing on its part, specifically, being able to decide if even certain light requests should be directly sent to an EC2 instance. In addition, there is room for improvements on the cost estimation algorithm, either by optimizing how KNN is performed or using a different algorithm entirely.

References

[1] R. Shea, J. Liu, E. C.-H. Ngai, and Y. Cui, "Cloud gaming: architecture and performance," *IEEE network*, vol. 27, no. 4, pp. 16–21, 2013.

[2] H. Dubey, "Efficient and accurate knn based classification and regression," *A Master Thesis Presented to the Center for Data Engineering, International Institute of Information Technology*, vol. 500, no. 32, 2013.